

TD Chapitre 1 : Représentation Numérique

Exercices Forensiques

Dr. El Hadji Bassirou TOURÉ
Département de Mathématiques et Informatique
Faculté des Sciences et Techniques
Université Cheikh Anta Diop de Dakar

1 Conversions de Base

Exercice 1 : Conversions binaire $\leftrightarrow$ hexadécimal $\leftrightarrow$ décimal

Compléter le tableau suivant (toutes les valeurs sont non signées sur 8 bits) :

Binaire	Hexadécimal	Décimal
10110101 ₂		
	0x7F	
		200
11001010 ₂		
	0xE3	
		47

Exercice 2 : Conversion décimal $\rightarrow$ binaire

En utilisant la méthode des divisions successives, convertir les nombres suivants en binaire sur 8 bits :

- a) 73₁₀
- b) 156₁₀
- c) 255₁₀

Montrer toutes les étapes de calcul.

2 Complément à Deux

Exercice 3 : Interprétation signed/unsigned

Le dump mémoire suivant montre 8 octets consécutifs à partir de l'adresse 0x1000 :

0x1000:	7F	80	FF	00	FE	01	C8	38
---------	----	----	----	----	----	----	----	----

Pour chaque octet, donner sa valeur interprétée comme :

- unsigned char (0 à 255)
- signed char (-128 à 127)

Compléter le tableau :

Adresse	Hex	unsigned char	signed char
0x1000	0x7F		
0x1001	0x80		
0x1002	0xFF		
0x1003	0x00		
0x1004	0xFE		
0x1005	0x01		
0x1006	0xC8		
0x1007	0x38		

Exercice 4 : Négation en complément à deux

Calculer la négation des nombres suivants sur 8 bits en utilisant la formule $-x = \sim x + 1$:

- $x = 42$ (soit 0x2A)
- $x = -1$ (soit 0xFF)
- $x = -128$ (soit 0x80)

Pour chaque cas, montrer :

- Le motif binaire de x
- Le motif binaire de $\sim x$
- Le motif binaire de $\sim x + 1$
- La valeur décimale du résultat

Exercice 5 : Plages et valeurs limites

- Donner les valeurs T_{min} et T_{max} pour `int16_t` (16 bits signés).
- Que vaut 0xFFFF interprété comme `int16_t` ?
- Que vaut 0x8000 interprété comme `int16_t` ?
- Que vaut 0x7FFF interprété comme `int16_t` ?

3 Forensics Mémoire

Exercice 6 : Analyse de dump mémoire (Little-Endian)

On observe le dump mémoire suivant sur une machine x86-64 (little-endian) :

0x2000:	34	12	00	00	FF	FF	FF	FF
0x2008:	FE	FF	00	00	80	00	00	00

Ce dump contient 4 variables de type `int32_t` (32 bits signés) aux adresses 0x2000, 0x2004, 0x2008, 0x200C.

- Rappeler l'ordre des octets en little-endian.

- b) Reconstituer la valeur hexadécimale 32 bits de chaque variable.
- c) Calculer la valeur décimale de chaque variable.

Exercice 7 : Détection de type

Le code suivant a été exécuté :

```
1 #include <stdint.h>
2 int8_t a = ???;
3 uint8_t b = ???;
4 int16_t c = ???;
5 uint32_t d = ???;
```

Le dump mémoire (little-endian) montre :

a (0x3000):	F5			
b (0x3001):	F5			
c (0x3002):	D2	FF		
d (0x3004):	00	01	00	00

Déterminer la valeur décimale de chaque variable.

4 IEEE 754 (Flottants)

Exercice 8 : Décomposition d'un float

Le nombre flottant float $x = -6.5f$ est stocké en mémoire. Sa représentation hexadécimale est 0xC0D00000.

- a) Convertir 0xC0D00000 en binaire (32 bits).
- b) Identifier les 3 champs : signe (1 bit), exposant (8 bits), fraction (23 bits).
- c) Calculer l'exposant réel E (rappel : biais = 127).
- d) Reconstituer la mantisse $M = 1.\text{fraction}$.
- e) Vérifier que $V = (-1)^s \times M \times 2^E = -6.5$.

Exercice 9 : Encodage IEEE 754

Encoder le nombre +9.75 en format IEEE 754 float (32 bits).

- a) Déterminer le signe.
- b) Convertir 9.75 en binaire fractionnaire.
- c) Normaliser sous la forme $1.xxx \times 2^n$.
- d) Calculer l'exposant biaisé.
- e) Écrire la fraction (sans le "1" implicite).
- f) Donner le résultat final en hexadécimal.

Exercice 10 : Valeurs spéciales

Donner la représentation hexadécimale (32 bits) des valeurs suivantes :

- a) +0.0
- b) -0.0
- c) $+\infty$
- d) $-\infty$

e) Un NaN (Not a Number)

Indice : pour $+\infty$, l'exposant vaut 0xFF et la fraction vaut 0x000000.

5 Pièges de Conversion

Exercice 11 : Conversions implicites

Prédire la sortie de chaque `printf` et expliquer pourquoi :

```
1 #include <stdio.h>
2 int main() {
3     // a) Comparaison signed/unsigned
4     int x = -1;
5     unsigned int y = 0;
6     printf("a) -1 < 0U: %d\n", x < y);
7
8     // b) Extension de signe
9     signed char sc = -10;
10    unsigned int ui = sc;
11    printf("b) ui = %u\n", ui);
12
13    // c) Troncature
14    int big = 0x12345678;
15    signed char small = big;
16    printf("c) small = %d (0x%02X)\n", small, (unsigned char)small);
17
18    // d) Comparaison avec constante unsigned
19    int i = -1;
20    if (i > sizeof(int))
21        printf("d) -1 > sizeof(int)\n");
22    else
23        printf("d) -1 <= sizeof(int)\n");
24
25    return 0;
26 }
```

Exercice 12 : Overflow et Undefined Behavior

Pour chaque code, indiquer si le comportement est :

- **D** : Défini (résultat prévisible)
- **UB** : Undefined Behavior

Et donner le résultat (si défini) ou expliquer le danger (si UB).

```
1 // a)
2 unsigned char a = 255;
3 a++;
4 printf("a = %d\n", a);
5
6 // b)
7 signed char b = 127;
8 b++;
9 printf("b = %d\n", b);
10
11 // c)
12 unsigned int c = 0;
13 c--;
14 printf("c = %u\n", c);
15
16 // d)
17 int d = 2147483647; // INT_MAX
18 d++;
19 printf("d = %d\n", d);
20
21 // e)
22 int e = -2147483648; // INT_MIN
23 int f = -e;
24 printf("f = %d\n", f);
```

Exercice 13 : Analyse de bug

Le code suivant contient un bug classique. Identifier le problème et proposer une correction.

```
1 #include <string.h>
2 #include <stdlib.h>
```

```

3
4 char* safe_copy(const char* src, size_t max_len) {
5     size_t len = strlen(src);
6     if (len - max_len > 0) { // Bug ici!
7         return NULL; // Source trop longue
8     }
9     char* dst = malloc(len + 1);
10    if (dst) strcpy(dst, src);
11    return dst;
12 }

```

- a) Quel est le type de `len - max_len` ?
- b) Que se passe-t-il si `len = 5` et `max_len = 10` ?
- c) Proposer une correction.